

Compression of NMT Models Via Pruning

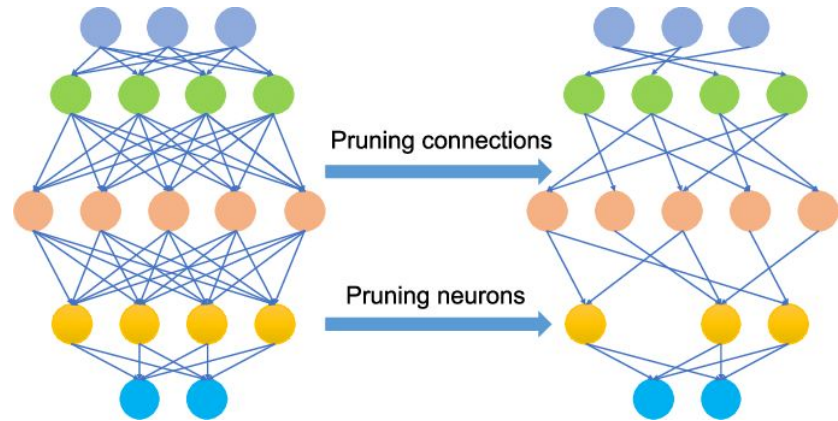
Team - VAKgpt

What is Pruning?

Pruning is the practice of removing parameters (which may entail removing individual parameters, or parameters in groups such as by neurons) from an existing network.

Types of Pruning

- Structured pruning involves pruning entire neuron, filters etc.
- Unstructured Pruning involves pruning individual parameters based on some saliency criteria.



Why do we need pruning?

- **Efficiency Boost:** Pruning trims unnecessary weights in over-parameterized neural networks, enhancing efficiency, decreasing floating operations per second.
- **Suboptimal Optimisation:** Pruning prevents getting stuck in suboptimal local minima, promoting robust feature learning.
- **Generalization Improvement:** Pruning improves neural network generalization by removing irrelevant weights, optimizing for specific tasks.

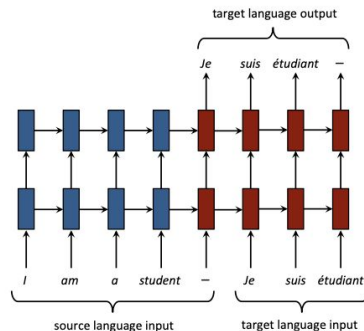
Our Task

Neural Machine Translation: It involves translating a sentence X of one language (source language) to sentence y of another language (target language).

A basic form of NMT consists of two components:

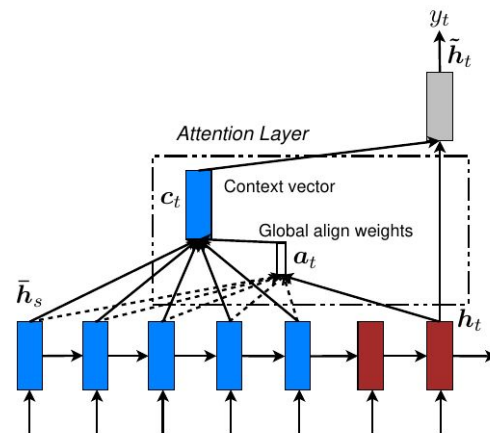
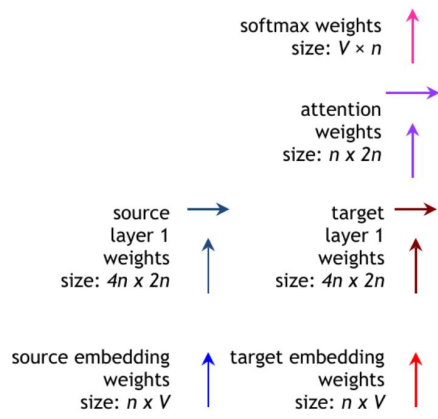
- (a) an encoder which computes a representation s for each source sentence
- (b) a decoder which generates one target word at a time and hence decomposes the conditional probability as:

$$\log p(y|x) = \sum_{j=1}^m \log p(y_j | y_{<j}, s)$$



Our Model

Our model's architecture is based on the model by (Luong et al. 2015) which introduced global and local dot product attentions.



Attention in model

- Calculate the alignment score of current target hidden state with each source hidden state .
- A global context vector is then computed as the weighted average over all the source states.
- Combine the information from source context and target hidden layer to produce an attentional hidden state .

$$\begin{aligned} \mathbf{a}_t(s) &= \text{align}(\mathbf{h}_t, \bar{\mathbf{h}}_s) \\ &= \frac{\exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_s))}{\sum_{s'} \exp(\text{score}(\mathbf{h}_t, \bar{\mathbf{h}}_{s'}))} \end{aligned}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c[\mathbf{c}_t; \mathbf{h}_t])$$

More details:

- We use only 1-layer of LSTM (as compared to original 4) stacked in decoder and encoder.
- We use IWSLT 2014 German-English dataset (150K training sentences).
- Both these points bring training time from few days to few hours, speeding our experiments.
- All the experiments are performed on 3 of the models and we average out the results

Our Model-Transformer

Encoder

- Stack of 6 identical layers .
- Each layer has:
 1. **Multi-head self-attention.**
 2. **Feed-forward network.**
 3. **Residual connections and Layer Normalization.**

Decoder

- Also has 6 layers.
- Each layer has:
 1. **Masked multi-head self-attention** (prevents cheating by lool
 2. **Multi-head attention over encoder outputs** (cross-attention
 3. **Feed-forward network.**
 4. **Residuals + Layer Norm.**

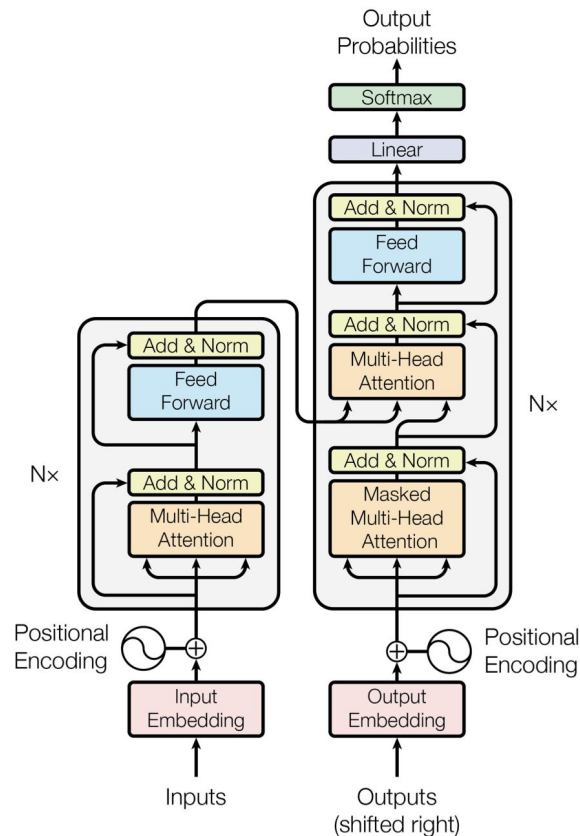
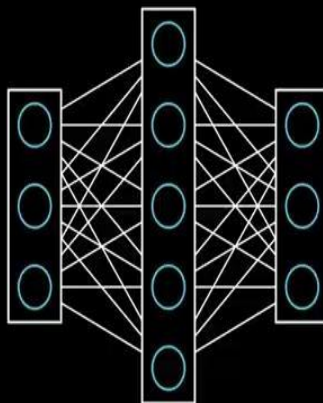


Figure 1: The Transformer - model architecture.

Pruning Schemes implemented in our paper (See et al. 2016)

- **Class-blind magnitude pruning:** Sort the parameters of whole model. Find the threshold for pruning such that $x\%$ of weights is pruned.
- **Class-uniform magnitude pruning:** Sort the parameters of each class of weights and within each class find the threshold for pruning such that $x\%$ of weights in all classes of weights are pruned.
- **Class-distribution magnitude pruning:** For each class c , weights with magnitude less than $\lambda \sigma_c$ are pruned. Here, σ_c is the standard deviation of that class and λ is a universal parameter chosen such that in total, $x\%$ of all parameters are pruned.

Class-Uniform Pruning



Weights 1

$$\begin{bmatrix} 0.8 & 0.5 & 0.7 & 0.5 & 0.1 \\ 0.7 & 0.6 & 0.6 & 0.5 & 0.9 \\ 0.4 & 0.5 & 0.3 & 0.1 & 0.2 \end{bmatrix}$$

Weights 2

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \\ 0.3 & 0.2 & 0.1 \\ 0.3 & 0.2 & 0.1 \\ 0.3 & 0.2 & 0.1 \\ 0.3 & 0.2 & 0.1 \end{bmatrix}$$

Retraining and Sparse from Begin(SFB)

- **What is retraining?**

We continue to train the remaining weights by minimal change in hyperparameters, but maintain the sparse structure introduced by pruning.

- **How does retraining help?**

During pruning since we remove the connections our performance will decrease. Now by retraining the network is encouraged to rely on the remaining important features and adjust their weights to capture the relevant patterns in the data.

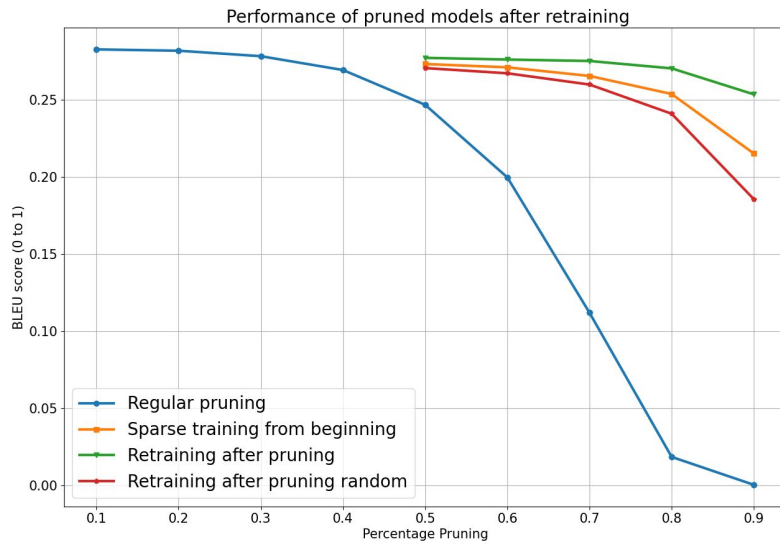
- **Training with sparse form from begin**

We prune our model maintain the sparsity structure and reinitialise the weights for training.

Effect of Retraining and Sparse from begin(SFB)

Retraining improves performance on high pruning %age and acts as regularizer for low %ages.

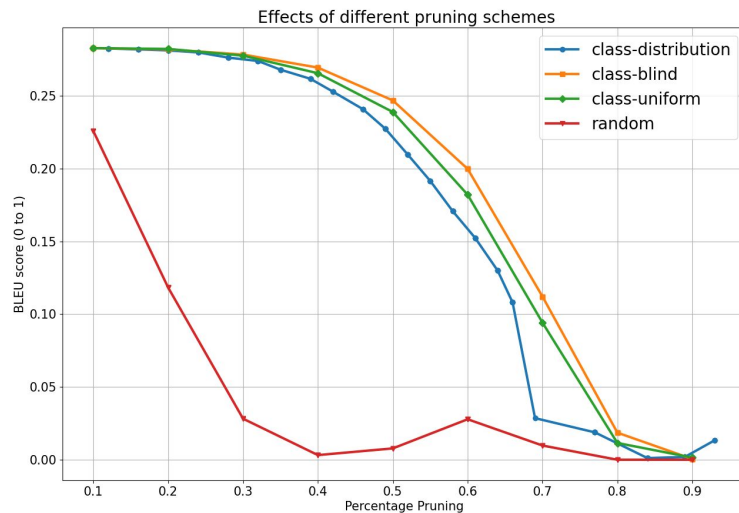
Sparsity structure alone contains useful information about redundancy



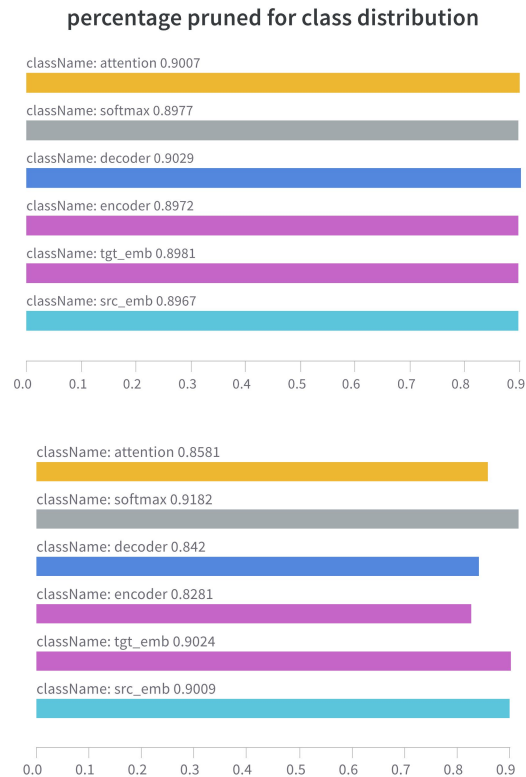
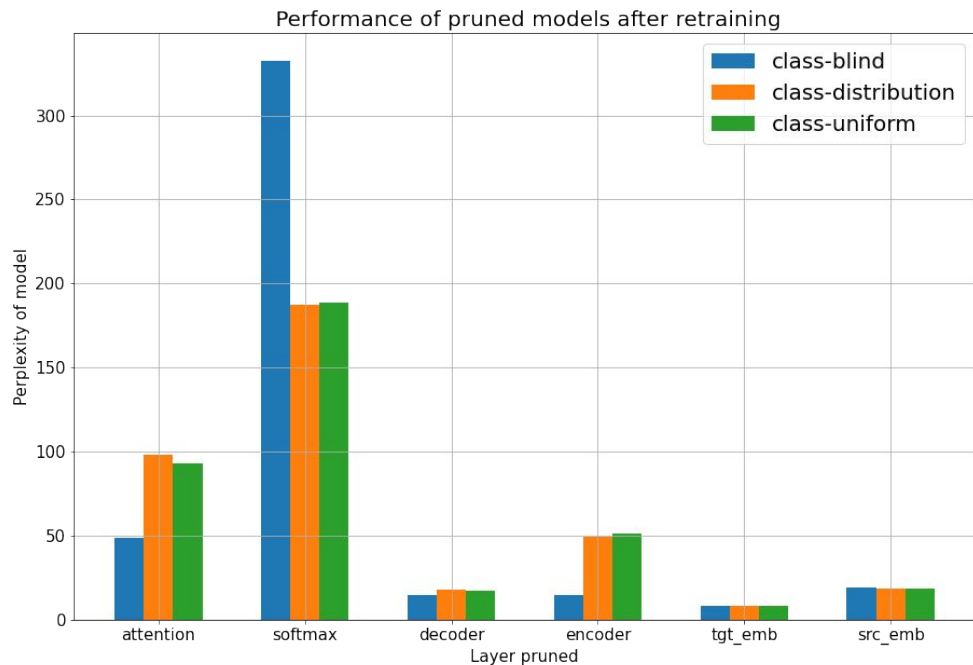
Comparison of these pruning schemes with baseline

Class blind is better - Adheres to the principle that pruning weights is least damaging when those weights are small regardless of their locations in the architecture.

Other methods prune weights in classes which have higher magnitudes and can be more important.



Perplexity Change with pruning of each class of weights



What we have covered till now?

Pruning Types

- Class-blind magnitude pruning
- Class-uniform magnitude pruning
- Class-distribution magnitude pruning
- Random Pruning

Experiments

- Bleu score of different pruning schemes before and after retraining
- Perplexity change with each weight class
- Percentage Pruned within each weight class

Shortcomings of Magnitude Pruning

- Magnitude of weights as a saliency criterion is only a heuristic to prune the weights. There can be more explainable, rigorous and principled methods.
- Prune-retrain cycles which are used to get the optimal results are costly, require additional hyperparameters (learning rate, no of epoch etc.) and may have architectural constraints.



Going beyond magnitude pruning !!
:)



SNIP: Single Shot Network Pruning based on connection sensitivity (Lee et al. 2019)

It prunes the connections at **initialization (before training)** and training normally. It identifies saliency criterion based on connection sensitivity that identifies structurally important connections in the network for the given task before training starts.

Why is there even a need for pruning before training?

This eliminates the need for both retraining and the complex pruning schedule while making it *robust to architecture variations*.

More details on SNIP

Saliency criterion: Connection sensitivity involves removing the connection and observing the change in the network's output (loss function value) compared to the original network.

$$\begin{aligned} \min_{\mathbf{c}, \mathbf{w}} L(\mathbf{c} \odot \mathbf{w}; \mathcal{D}) &= \min_{\mathbf{c}, \mathbf{w}} \frac{1}{n} \sum_{i=1}^n \ell(\mathbf{c} \odot \mathbf{w}; (\mathbf{x}_i, \mathbf{y}_i)) , \\ \text{s.t. } \mathbf{w} &\in \mathbb{R}^m , \\ \mathbf{c} &\in \{0, 1\}^m, \quad \|\mathbf{c}\|_0 \leq \kappa , \end{aligned}$$

Ideally, we would want to remove each weight $\mathbf{w}$ once (by setting some mask $\mathbf{c} = 0$), to check for effect on loss:

$$\Delta L_j(\mathbf{w}; \mathcal{D}) = L(\mathbf{1} \odot \mathbf{w}; \mathcal{D}) - L((\mathbf{1} - \mathbf{e}_j) \odot \mathbf{w}; \mathcal{D}) ,$$

More details on SNIP

$$g_j(\mathbf{w}; \mathcal{D}) = \left. \frac{\partial L(\mathbf{c} \odot \mathbf{w}; \mathcal{D})}{\partial c_j} \right|_{\mathbf{c}=\mathbf{1}}$$

Notice, that it is gradient with respect to mask, and not original weights.

This approximates the effect of change of loss if some mask is made 0, or equivalently a respective weight pruned.

Effect of Initialisation

Initialisation matters a lot since the connection sensitivity is performed on untrained weights.

It should be variance scaled for the method to not depend on architecture variations.

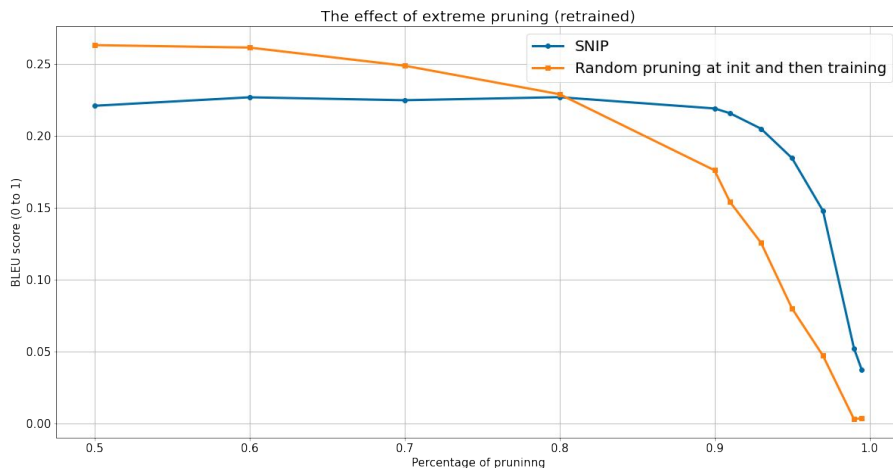
initialisation	percentage	bleu
kaming_normal	90	0.2001
uniform	90	0.1518
kaming_uniform	90	0.1929

Comparison with Random baseline

SNIP performs better at more extreme pruning percentage.

Another thing to notice is that original paper tested SNIP only on CNNs and RNNs **for image classification** (RNNs too).

In our **NMT** task, we find that performance is not always optimal.



Problems With SNIP

- Connection sensitivity is suboptimal as a criterion because the gradient of each weight might change dramatically after pruning due to complicated interactions between weights.
- Since SNIP only considers the gradient for one weight in isolation, it could remove connections that are important to the flow of information through the network.

Optimal Brain Damage (LeCun et al. 1989)

Using Taylor's expansion we can approximate change in loss, dL as:

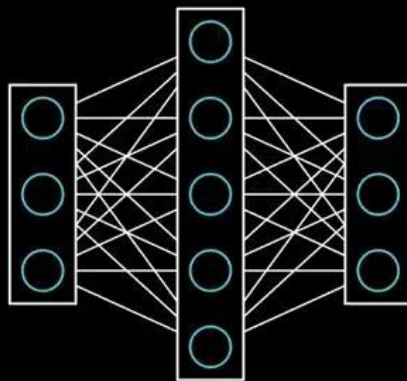
$$dL = \sum_i \frac{\partial L}{\partial w_i} dw_i + \frac{1}{2} \sum_i \left(\frac{\partial^2 L}{\partial w_i^2} \right) (dw_i)^2 + \frac{1}{2} \sum_{i \neq j} \frac{\partial^2 L}{\partial w_i \partial w_j} dw_i dw_j + O(n^3) \quad (1)$$

Now since the model has converged, the first term is zero and we ignore the interaction and higher order terms what we finally get is:

$$dL = \sum_i \left(\frac{\partial^2 L}{\partial w_i^2} \right) (dw_i)^2$$

Second derivative of the loss functions with respect to parameters is used as a saliency measure.

OBD Pruning



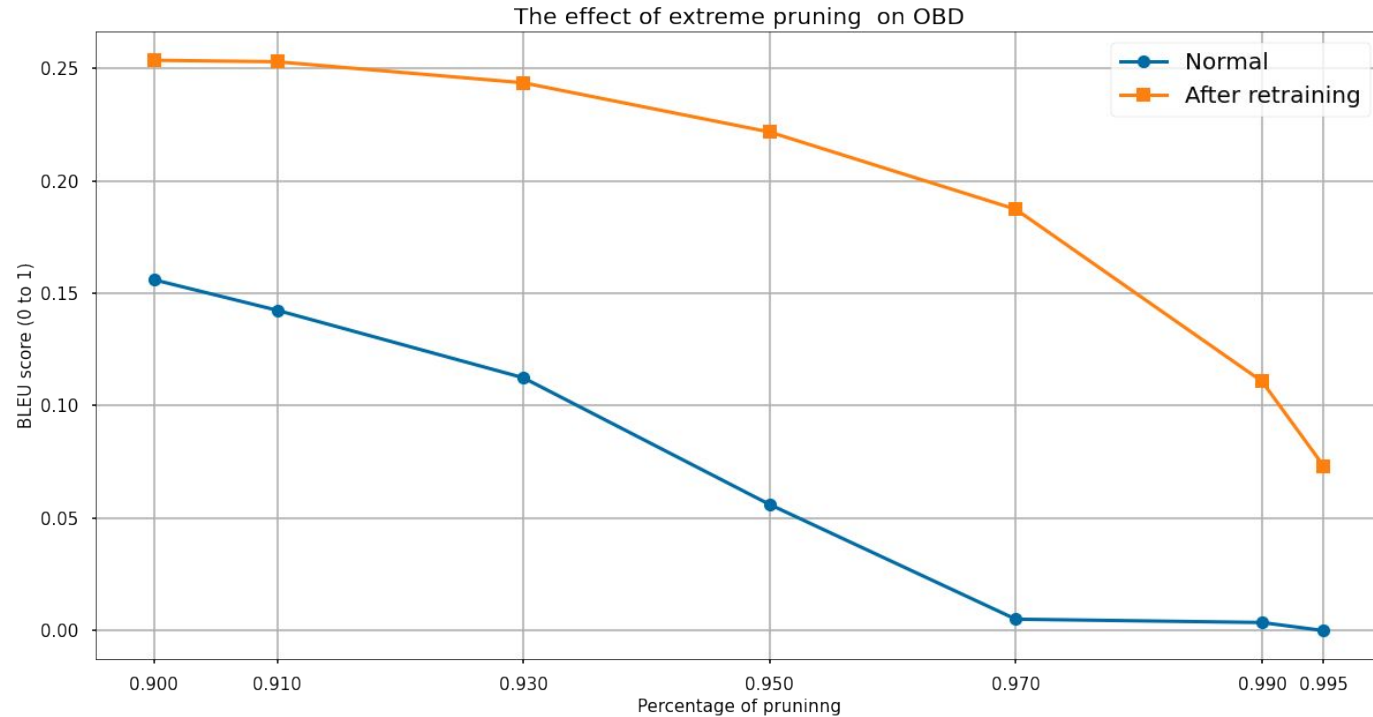
Weights 1

$$\begin{bmatrix} 0.44 & 0.61 & 0.53 & 0.5 & 0.49 \\ 0.9 & 0.57 & 0.6 & 0.6 & 0.7 \\ 0.81 & 0.5 & 0.7 & 0.5 & 0.8 \end{bmatrix}$$

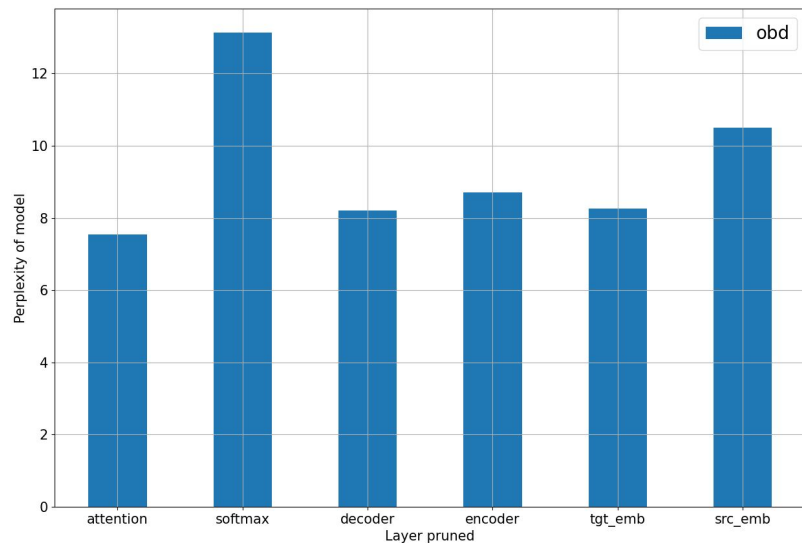
Weights 2

$$\begin{bmatrix} 0.71 & 0.32 & 0.8 \\ 0.7 & 0.73 & 0.41 \\ 0.7 & 0.5 & 0.51 \\ 0.5 & 0.72 & 0.91 \\ 0.3 & 0.52 & 0.9 \end{bmatrix}$$

Effect of extreme pruning percentages on OBD



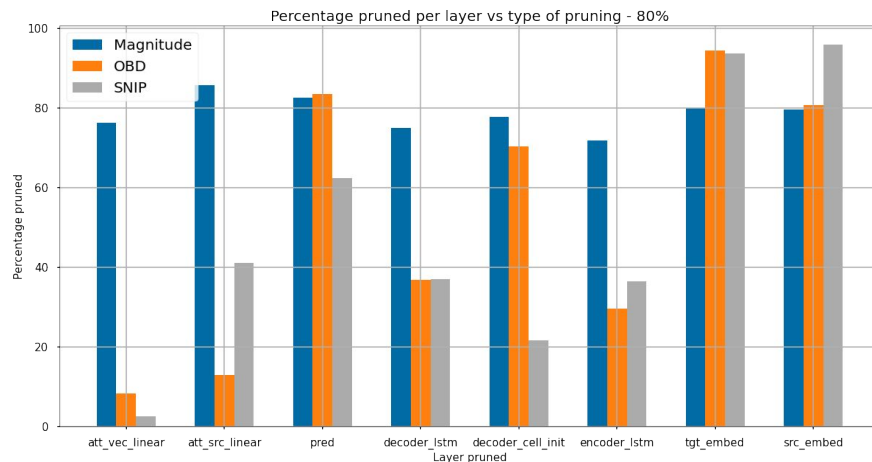
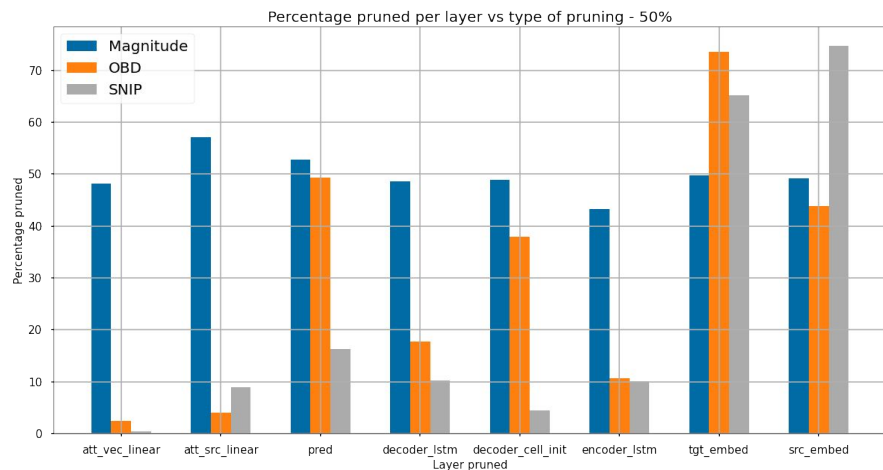
Perplexity Change by pruning each class of weights



Comparisons Between All Pruning Schemes Seen Till Now

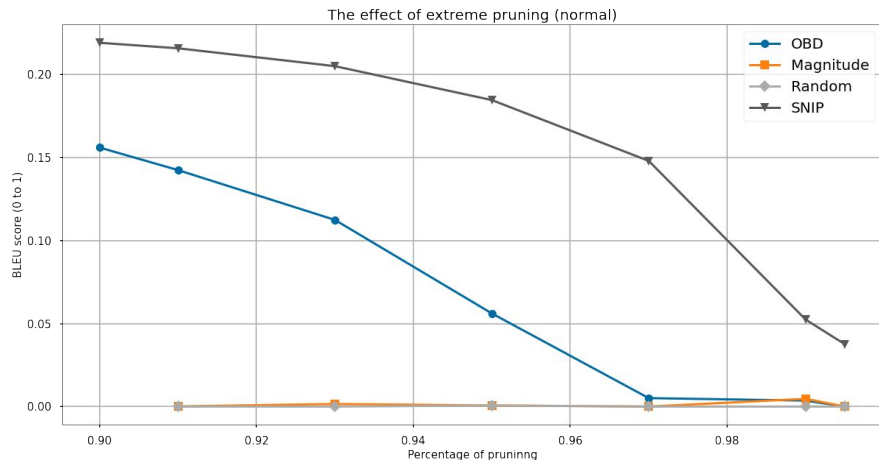
We performed various experiments to compare the pruning schemes in context to find their strengths more in view of our task

Percentage Pruned vs class type pruned for 50 and 80 percent pruning

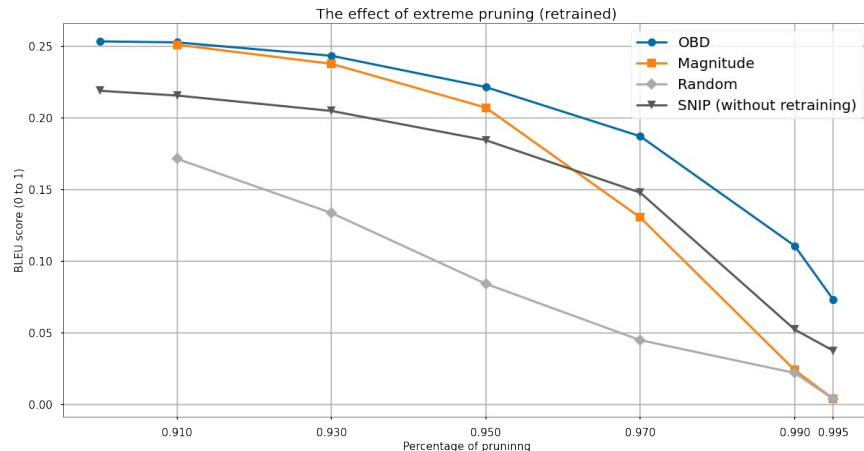


OBD and SNIP remove weights closer to initial lvls. This is actually good as later weights are considered to be more important.

Effect of extreme pruning percentages comparison



SNIP was pruned at initialisation, rest are pruned after training.



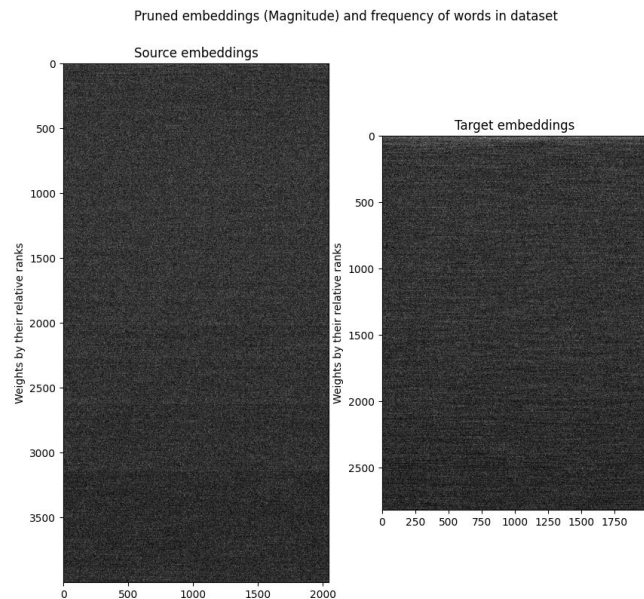
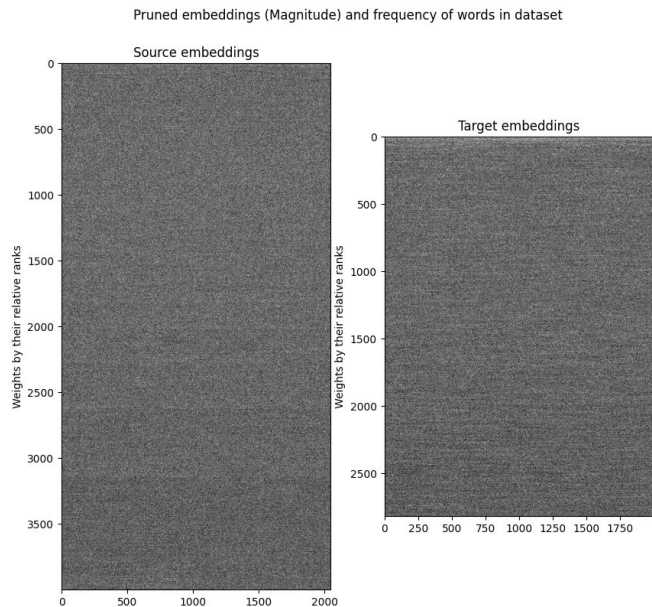
Top-3 pruning schemes are shown after retraining. SNIP is shown as usual.

Both graphs seem to confirm that SNIP is good at extreme levels of pruning.

Effect of pruning on word embeddings

Do words used less in dataset get pruned?

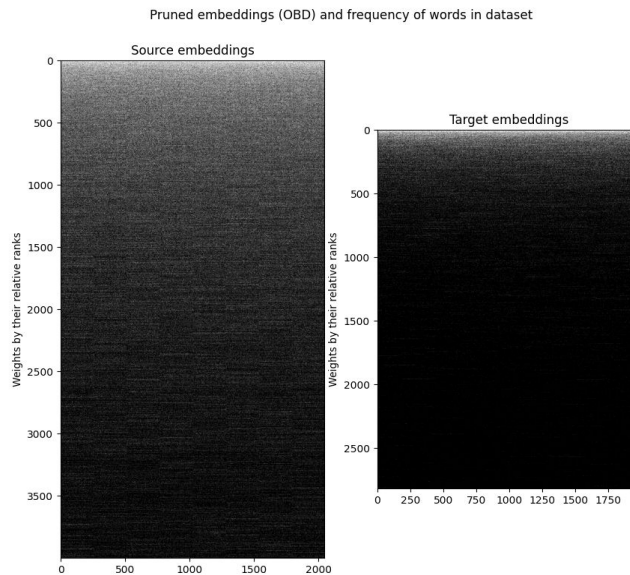
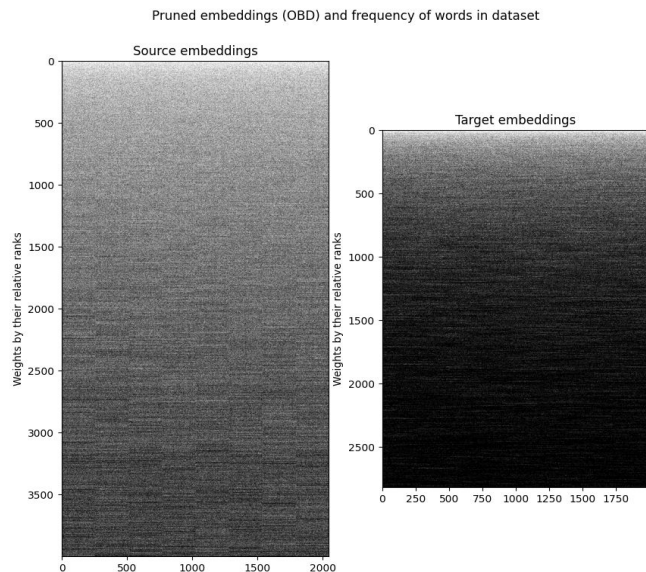
Magnitude pruning



Effect of pruning on word embeddings

Do words used less in dataset get pruned?

OBD pruning

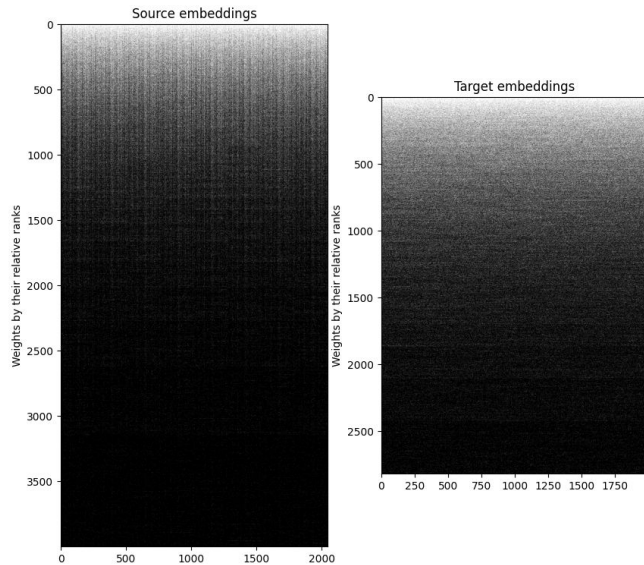


Effect of pruning on word embeddings

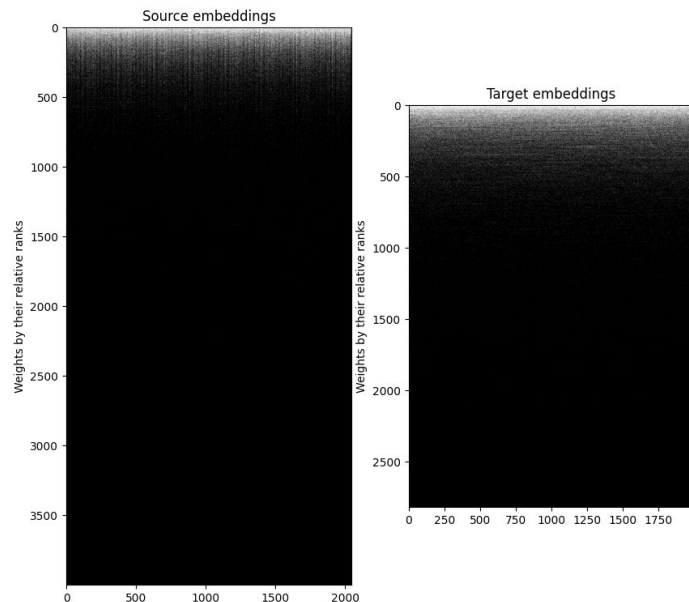
Do words used less in dataset get pruned?

SNIP pruning

Pruned embeddings (SNIP) and frequency of words in dataset



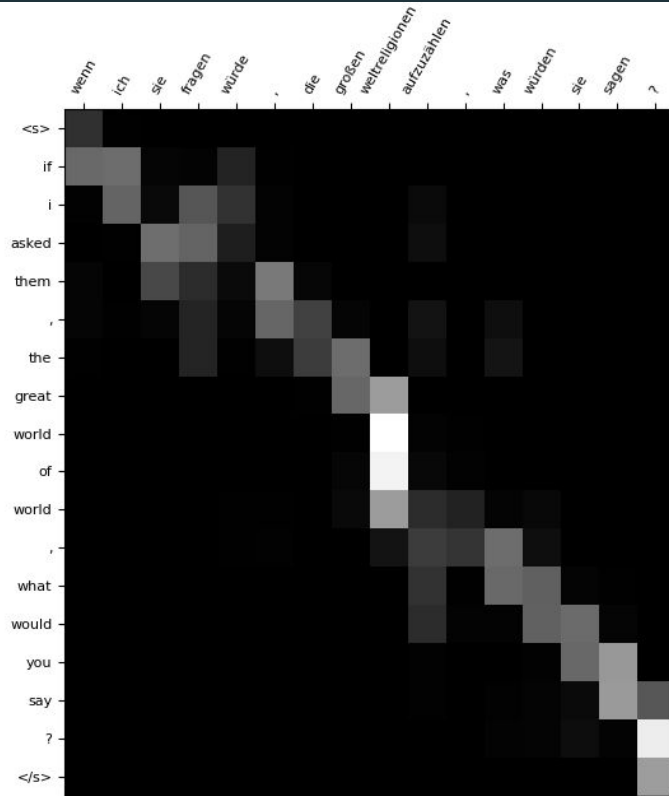
Pruned embeddings (SNIP) and frequency of words in dataset



Alignment of Attention Weights

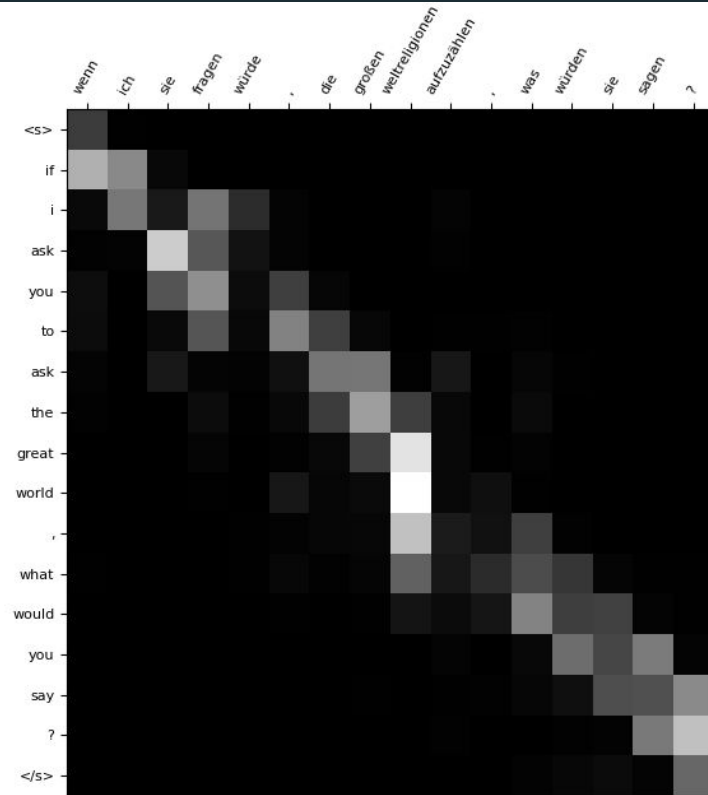
Source: wenn ich sie fragen würde , die großen weltreligionen aufzuzählen , was würden sie sagen ?

Target: if i asked you to name the great religions of the world , what would you say ?



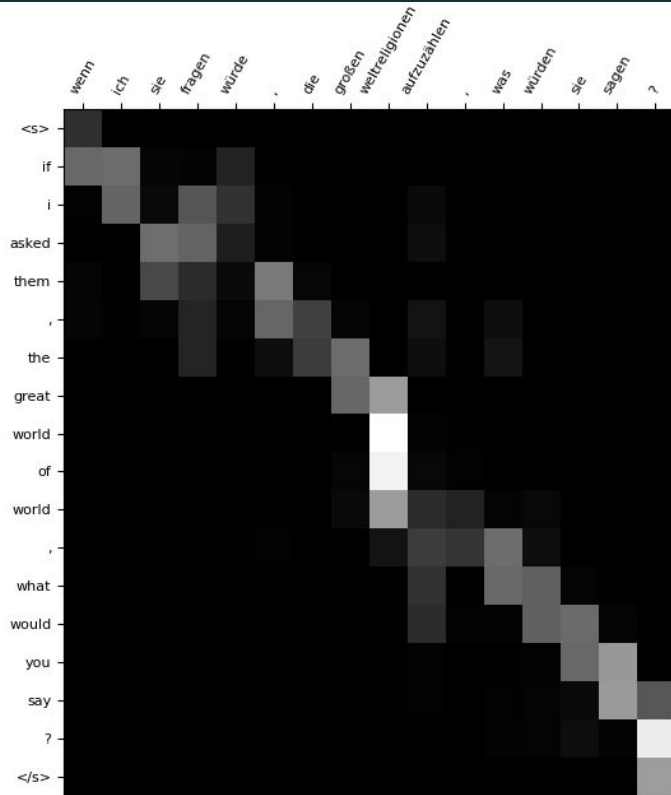
Alignment of source and target sentences

Predicted: If i asked them , the great world of world , what would you say ?



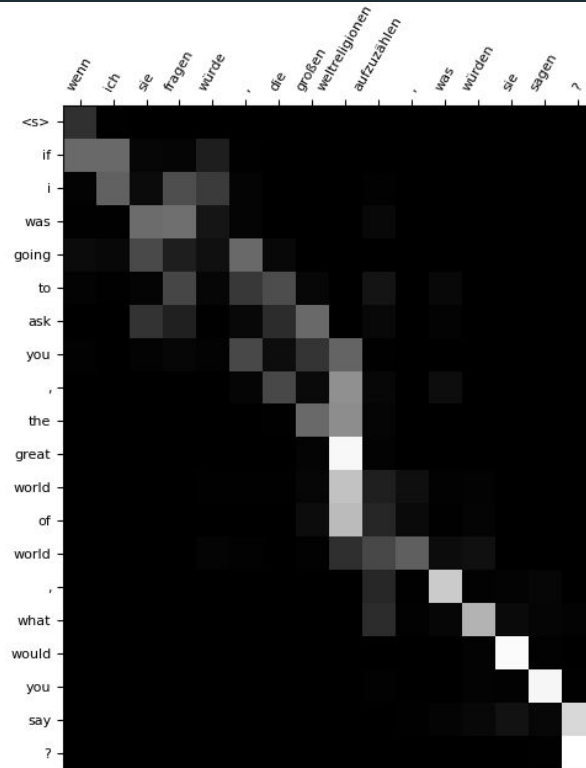
Alignment of source and target sentences for 50 globally pruned model

Predicted: If i ask you to ask the great world , what would you say ?



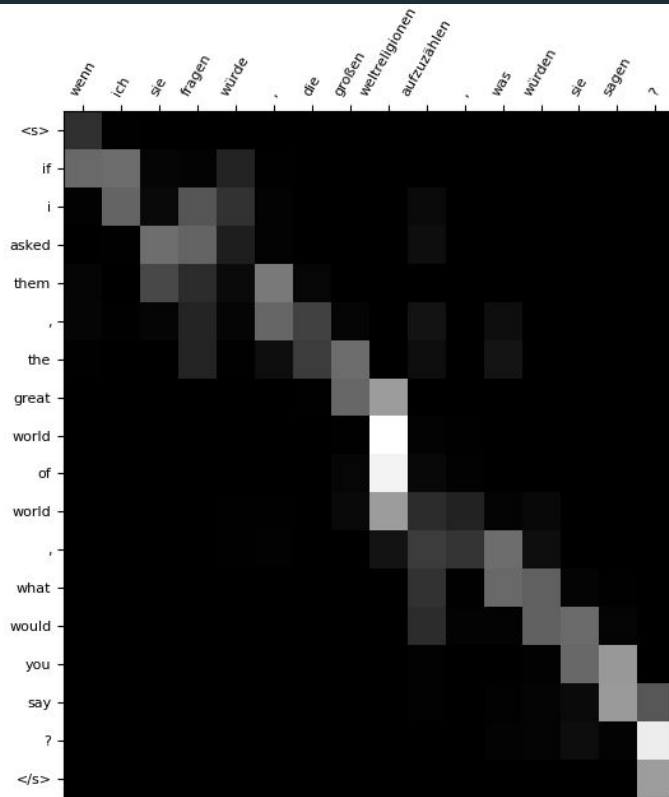
Alignment of source and target sentences

Predicted: If i asked them , the great world of world , what would you say ?



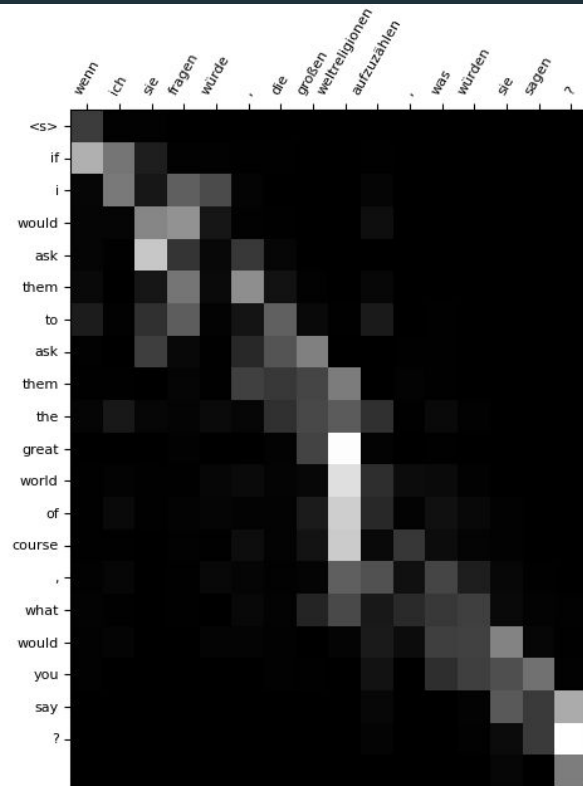
Alignment of source and target sentences for 50 OBD pruned model

Predicted: If i would ask them to ask them the great world of world, what would you say ?



Alignment of source and target sentences

Predicted: If i asked them , the great world of world , what would you say ?



Alignment of source and target sentences for 90 class-blind with retraining

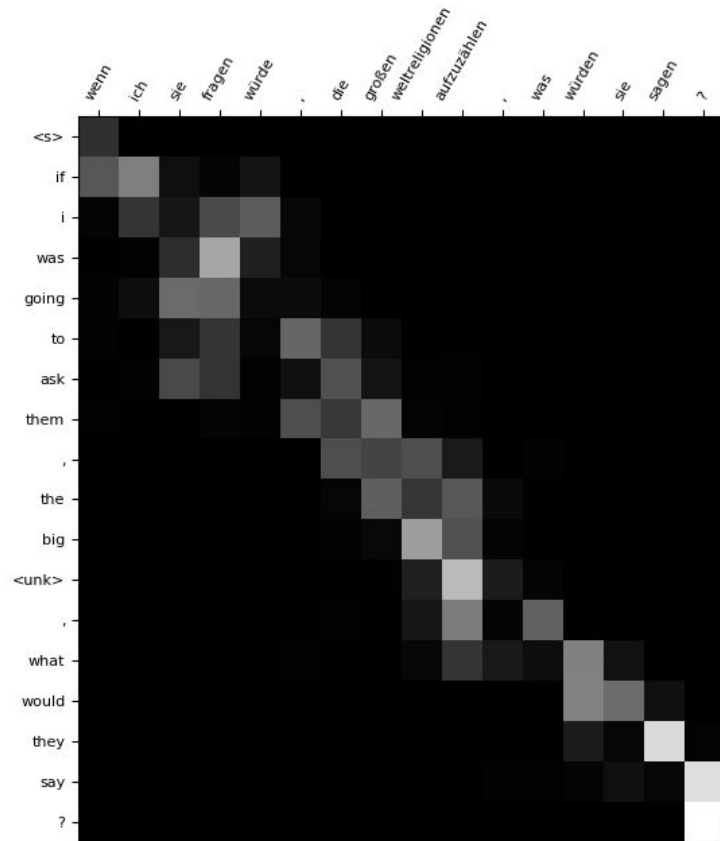
Predicted: If i would ask them to ask them the great world of course , what would you say ?

OBD With 90% without retraining

Source: wenn ich sie fragen würde , die
großen weltreligionen aufzuzählen ,
was würden sie sagen ?

Target: if i asked you to name the great
religions of the world , what would you
say ?

Predicted: If i was going to ask them ,
the big <unk> , what would they say ?



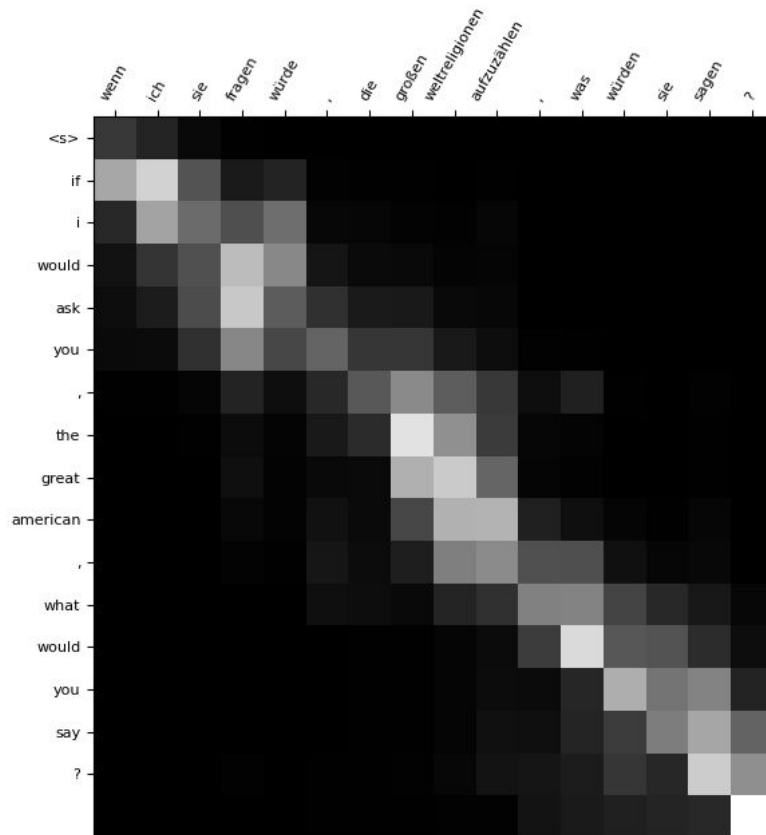
Alignment of source and target sentences for 90 obd without retraining

Snip with 90% Pruning

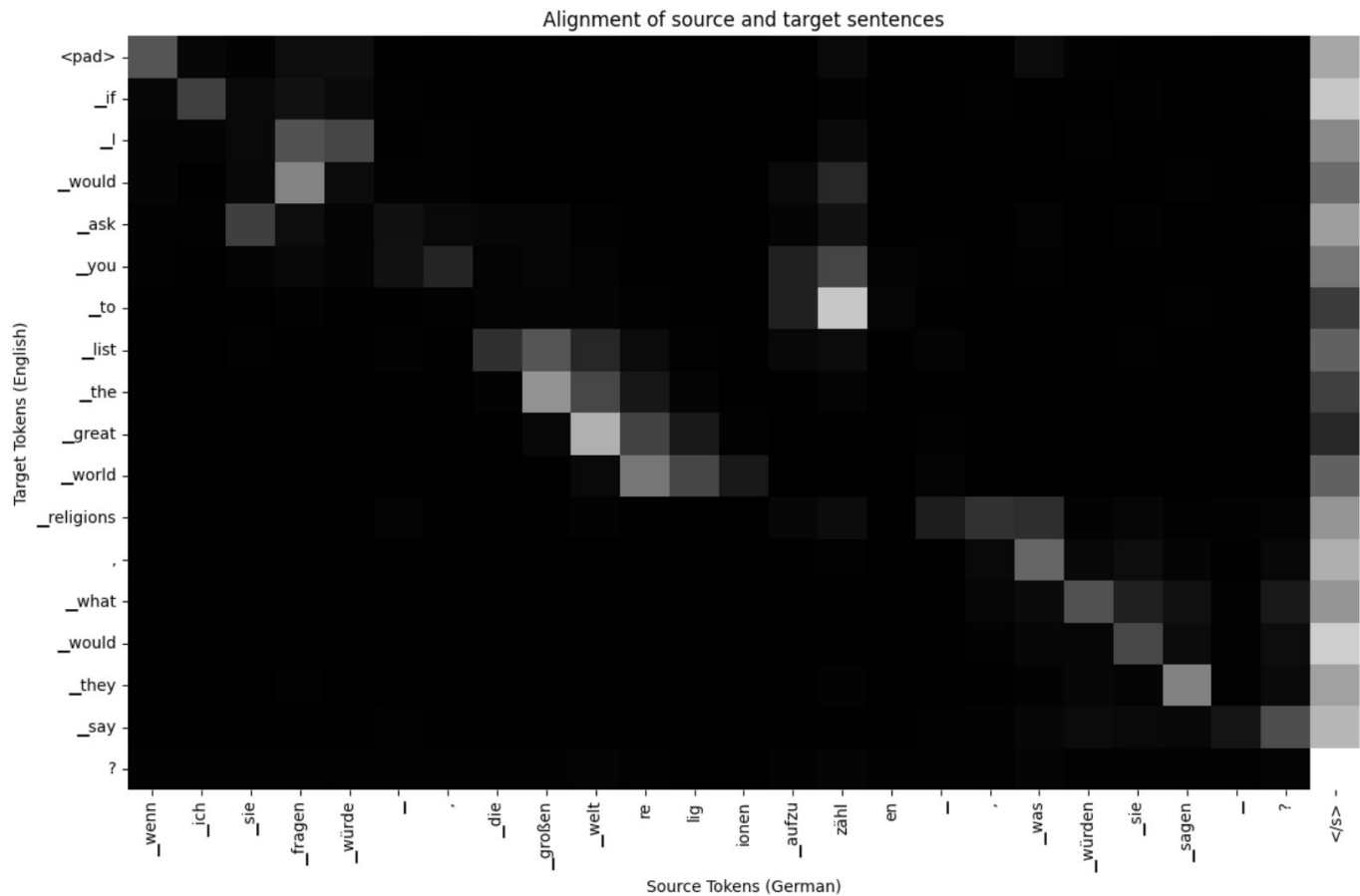
Source: wenn ich sie fragen würde , die großen weltreligionen aufzuzählen , was würden sie sagen ?

Target: if i asked you to name the great religions of the world , what would you say ?

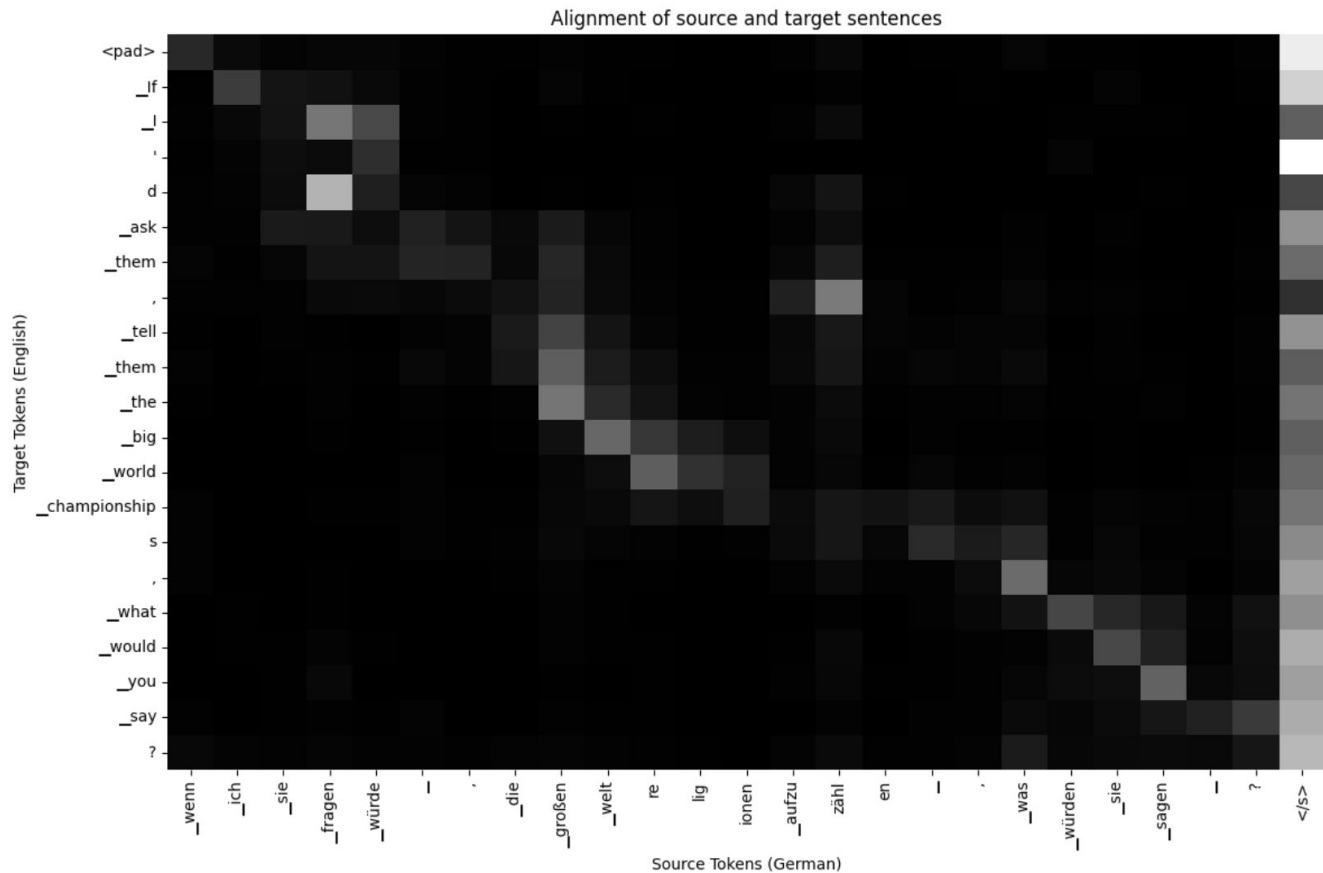
Predicted: If i would ask you , the great american , what would you say ?



Alignment of source and target sentences for 90 snip



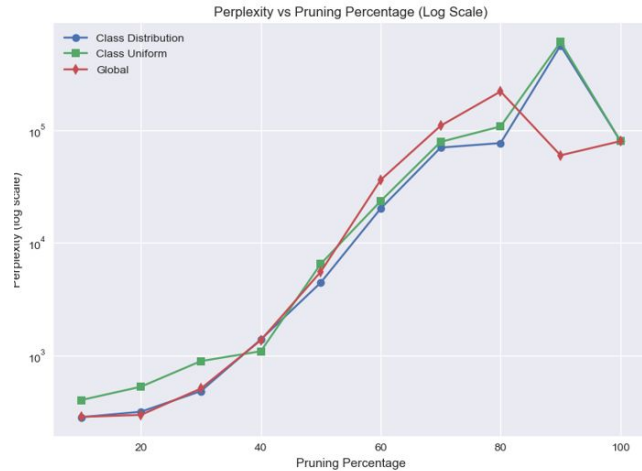
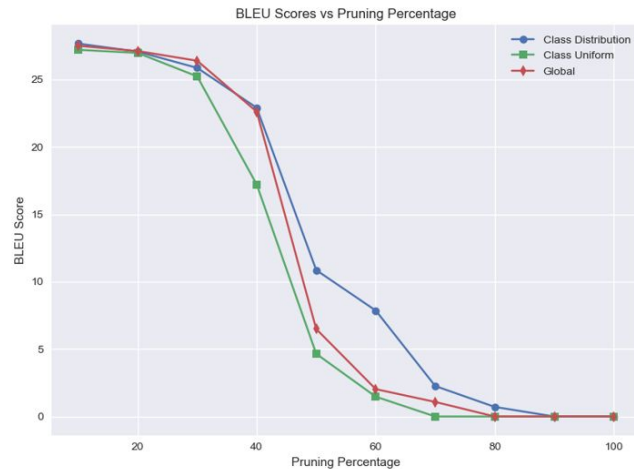
Pruned 30%: if I would ask you to list the great world religions, what would they say?



Pruned 50%: If I'd ask them, tell them the big world championships, what would you say?

Transformer

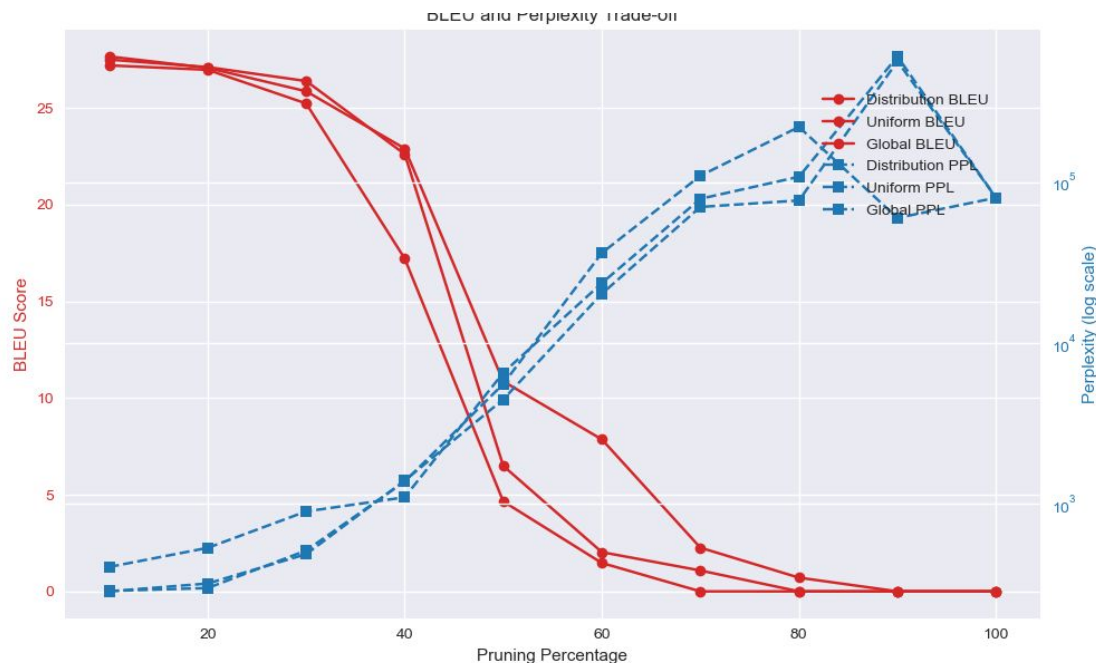
LSTMs process data sequentially, with less rigid weight specialization. **Class-blind pruning** treats all weights equally, avoiding over-pruning any single "important" part of the sequence. Transformers, however, rely on targeted attention, so blind pruning risks cutting crucial connections, while class-aware methods (like distribution) protect them.



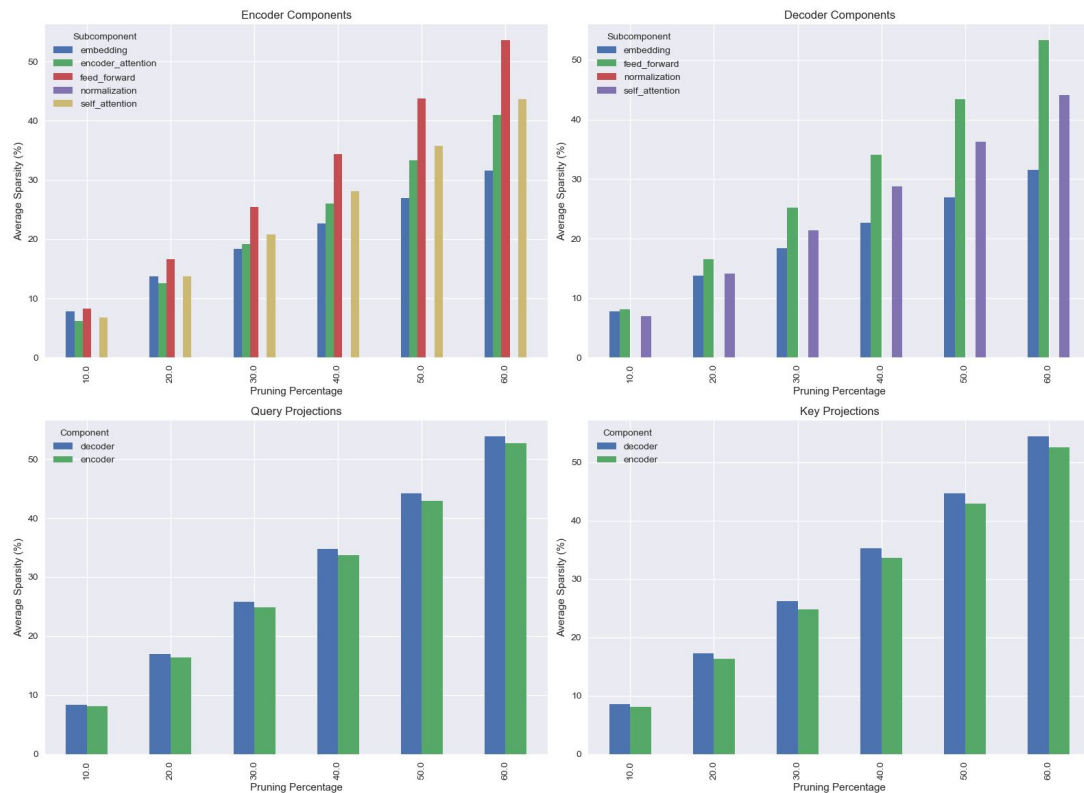
Bleu and perplexity tradeoff

Best pruning level appears to be around **20–30%**, where BLEU scores remain high (~26–27) and perplexity is still low. This suggests that mild pruning reduces model redundancy without hurting performance, offering a good compression-performance tradeoff.

From **40% onward**, BLEU drops sharply, while perplexity increases exponentially—especially past 50%, indicating the model is losing critical parameters.



Class distribution : Sparsity by component



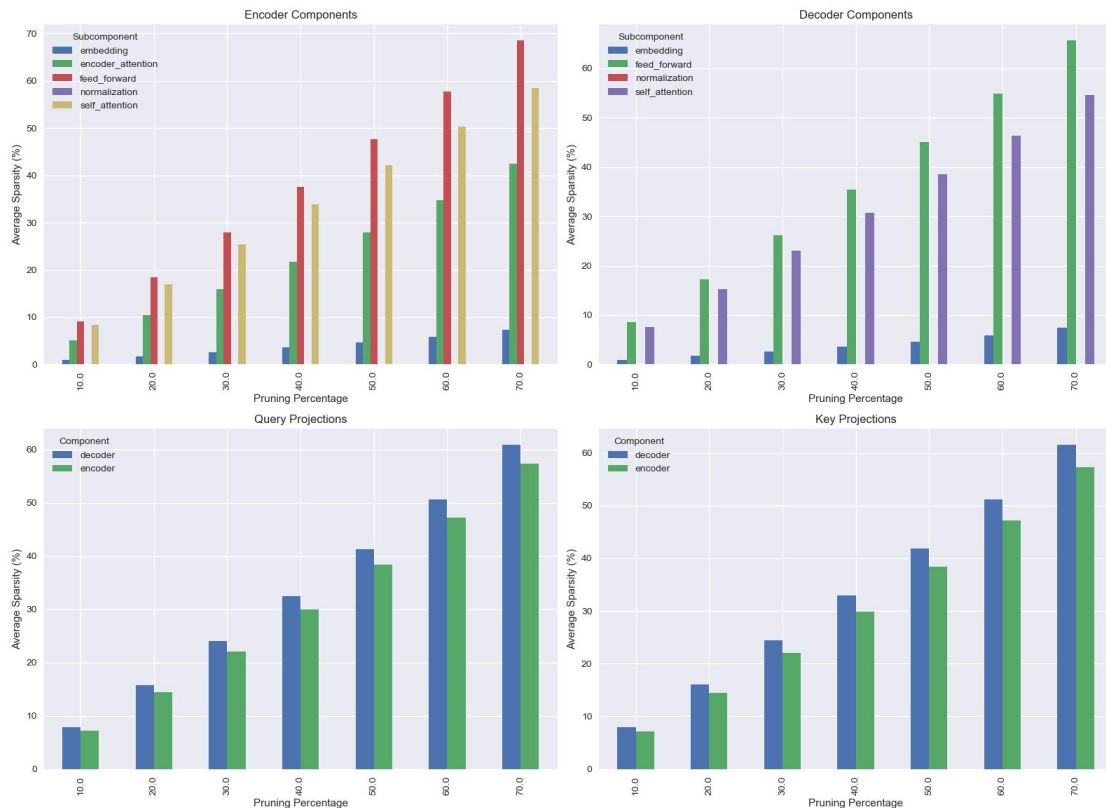
Observations :

Encoder Components (top-left): The **feed-forward** and **self-attention** layers show the highest sparsity at higher pruning levels (up to ~50%), indicating they are major contributors to the prunable parameter count. Embeddings and normalization layers remain less sparse, likely due to their smaller size or sensitivity to pruning.

Decoder Components (top-right): Similar to the encoder, **feed-forward layers** dominate in sparsity, followed by self-attention. Interestingly, **normalization** layers stay consistently less sparse, suggesting their role in model stability discourages heavy pruning.

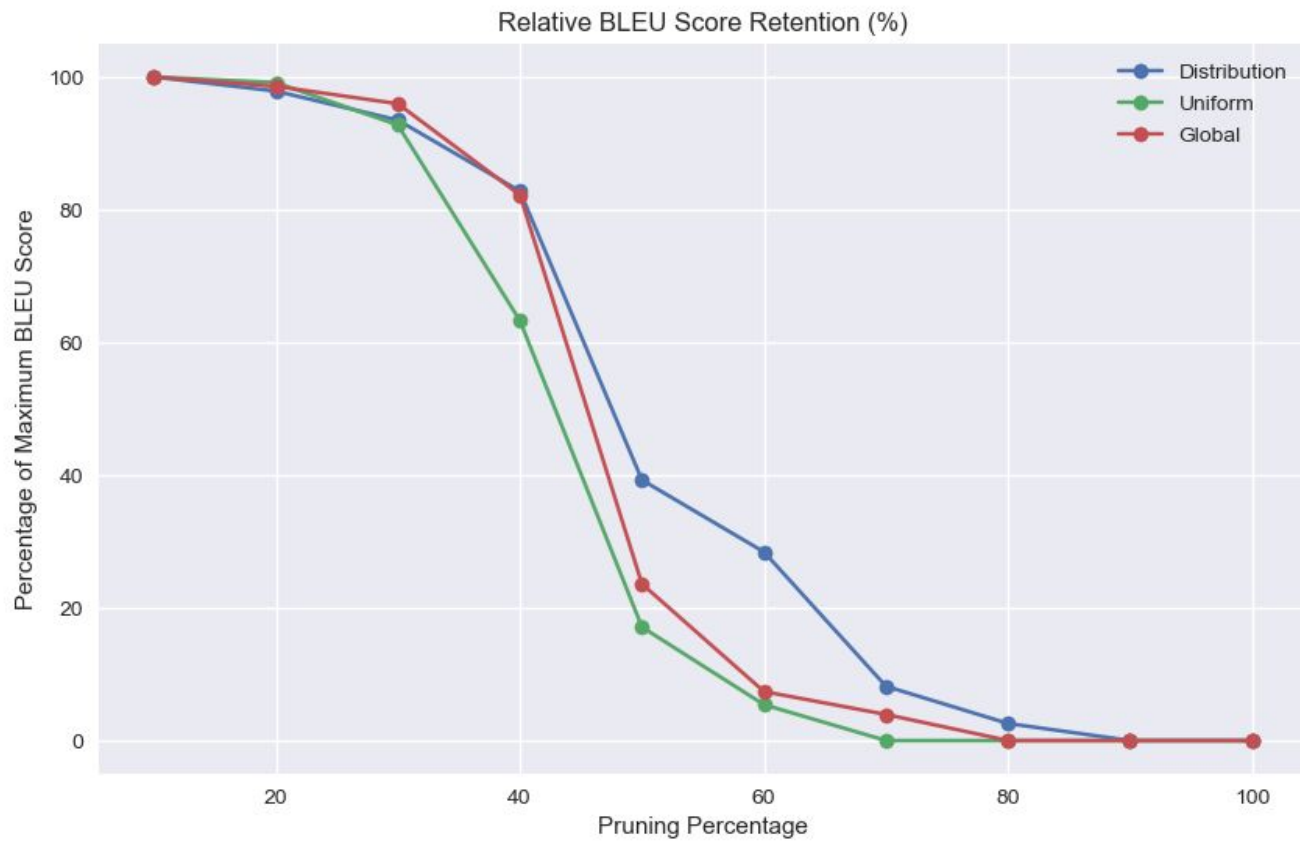
Query and Key Projections (bottom plots): Sparsity increases steadily in both encoder and decoder query/key projections, with both components showing similar trends. By 60% pruning, both query and key projections reach close to 50% sparsity, confirming these layers are heavily pruned—possibly due to redundancy in attention heads.

Global pruning : Sparsity by component



Observations:

- **Feed-forward layers** are still pruned the most, reaching over 60% sparsity at high pruning levels, similar to before.
- However, **normalization layers** and **embeddings** are now affected significantly more than in the previous setup. For example, embedding sparsity crosses 20% in some cases—unlike before, where it was largely preserved.
- This reflects the *structure-agnostic* nature of global pruning—it doesn't spare sensitive or smaller subcomponents, potentially hurting performance.



Low pruning (10–30%): All three methods retain >95% of the original BLEU score, showing that early pruning has minimal impact on translation quality. Distribution-based pruning slightly outperforms the others at 30%.

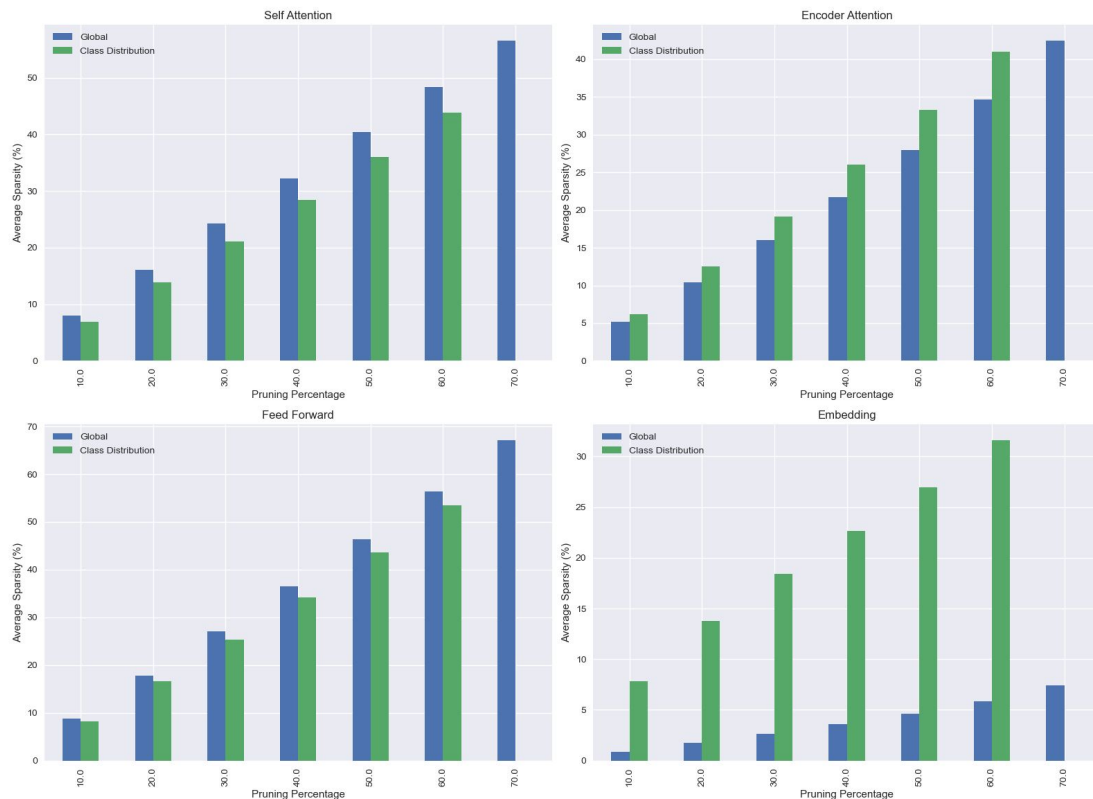
Mid pruning (40–60%):

- This is where divergence begins. **Distribution pruning** retains significantly more BLEU score (~40–30%) compared to **Global (~25%)** and especially **Uniform (~17%)**, which drops sharply.
- Suggests that **class-aware pruning** (distribution-based) is better at preserving functional capacity under medium sparsity.

High pruning (>70%):

- All methods converge to near-zero BLEU retention, indicating that aggressive pruning removes essential parameters regardless of strategy.
- **Uniform pruning** consistently underperforms throughout, likely due to blindly pruning equal amounts across layers without considering importance.

Sparsity comparison between methods



Observations :

1. Self-Attention

- Both strategies show a roughly linear increase in sparsity with pruning percentage.
- Global pruning consistently results in slightly higher sparsity than class-distribution pruning.
- Suggests that self-attention is aggressively pruned under global strategy, possibly overlooking class-wise significance.

2. Encoder Attention

- Class-distribution pruning yields higher sparsity than global at moderate-to-high pruning levels (40–70%).
- Indicates this method concentrates sparsity more heavily in this component, possibly identifying it as less critical compared to others.

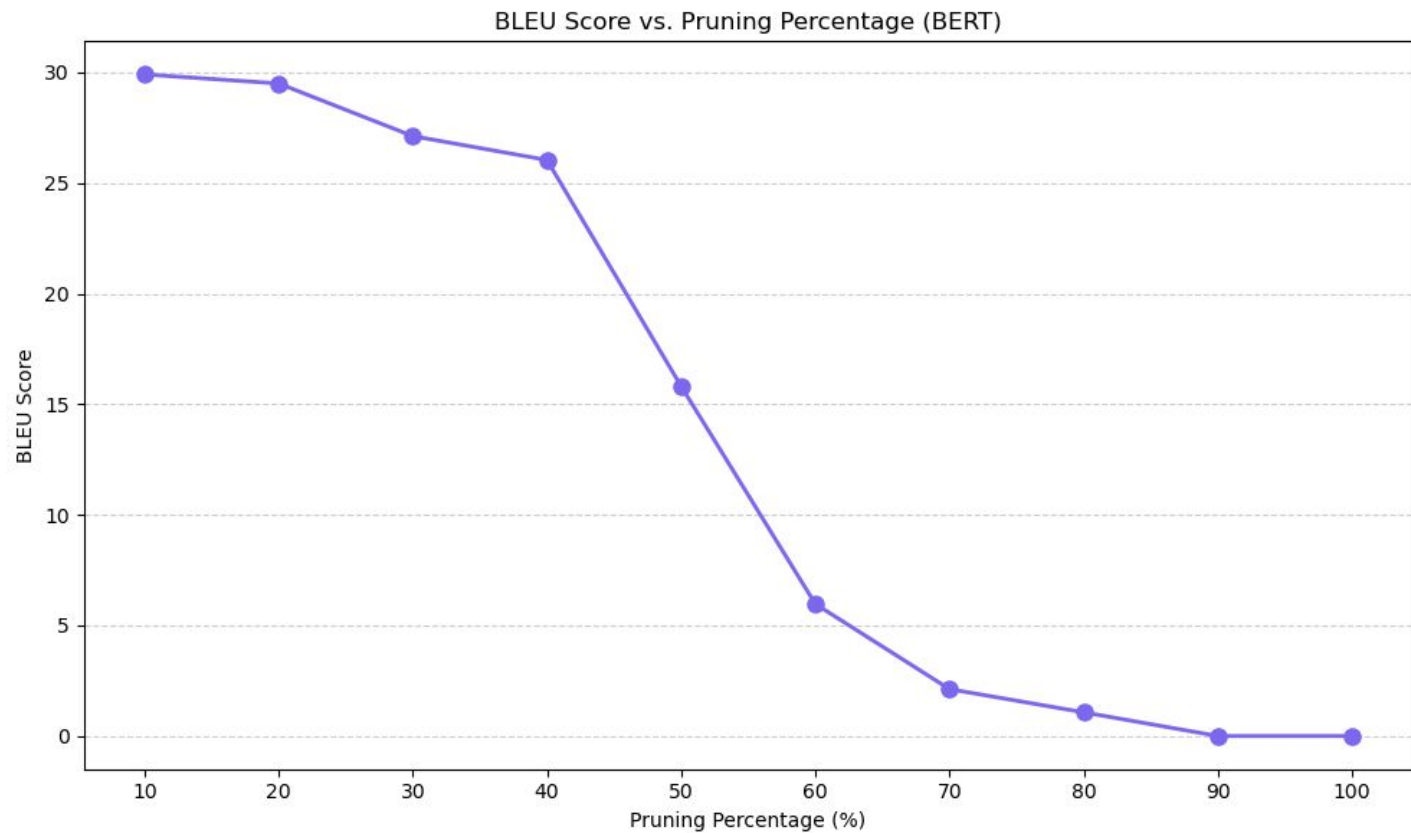
3. Feed Forward

- Sparsity under class-distribution exceeds that of global pruning especially at 40% and 60% levels.
- Shows that class-aware pruning targets feedforward layers more, aligning with known overparameterization in FFN sublayers of Transformers.

4. Embedding

- Striking divergence: class-distribution pruning yields significantly higher sparsity (~50%+ at 60–70%), while global pruning barely touches the embedding layer.
- Suggests global pruning over-protects embeddings, while class-aware strategy identifies and prunes redundant embedding dimensions more effectively

Bert



Final Conclusions based on our findings

- Class blind is best out of magnitude based prunings for LSTM , class Distribution for transformers
- Retraining can improve the model even if pruned randomly
- Gradient based methods prune initial layers more (due to less gradient flow to initial layers)
- OBD and SNIP perform good on extreme pruning percentages
- OBD performs really well even without retraining
- Performance of SNIP on NMT task as we found is suboptimal.
- We get best results on OBD out of all the pruning methods we tried

References

See, Abigail, Minh-Thang Luong, and Christopher D. Manning. "Compression of neural machine translation models via pruning." *arXiv preprint arXiv:1606.09274* (2016).

Luong, Minh-Thang, Hieu Pham, and Christopher D. Manning. "Effective approaches to attention-based neural machine translation." *arXiv preprint arXiv:1508.04025* (2015).

Lee, Namhoon, Thalaiyasingam Ajanthan, and Philip HS Torr. "Snip: Single-shot network pruning based on connection sensitivity." *arXiv preprint arXiv:1810.02340* (2018).

LeCun, Yann, John Denker, and Sara Solla. "Optimal brain damage." *Advances in neural information processing systems* 2 (1989).

Blalock, Davis, et al. "What is the state of neural network pruning?." *Proceedings of machine learning and systems* 2 (2020): 129-146.

Wang, Chaoqi, Guodong Zhang, and Roger Grosse. "Picking winning tickets before training by preserving gradient flow." *arXiv preprint arXiv:2002.07376* (2020).

Our codebase

Github Link: https://github.com/Vanshika-Dhingra/inlp_project

Report : <https://www.notion.so/PROJECT-REPORT-1ecd6cf8b18b80acb59efec1c248b01a>



Thank You !!